

Access using the Constructor - Encapsulation

Encapsulation

Bundling data and operations together while controlling the access to internal state.

class classname

DT Variable = Value

int a = ?

int b = ?

this.a = ? → construct

construct ←

{ In private
class

{ In public
class extend /
Implement
↓

getter - doesnot modify any
Values.

getter, setter

Call setter if you need to modify any Values.

Write a program - create a class name ^{as} student
include numbers, name, age.

```
class student {
```

```
    String name;
```

```
    int age;
```

```
    public void setName("String name") {  
        this.name = name;
```

```
    }  
    public void setAge("Int age") {  
        this.age = age;
```

```
    }  
    public void getName() {  
        return name;
```

```
    }  
    public void getAge() {  
        return age;
```

```

public void show() {
    System.out.println("name: " + name);
    System.out.println("age: " + age);
}

public static void main(String[] args) {
    Student s1 = new Student();
    s1.setName("Bob");
    s1.setAge(18);
    s1.show();
}
}

```

Abstraction

Abstraction is the process of eliminating not essential data type so that only the necessary details structural features of an entity remain visible to the caller. It establishes a clear boundary between the specification

Interfaces

An interface in java is pure structural construct that defines structural behaviours and implementing class must commit to, interfaces cannot be instantiated directly. A class can implement multiple interfaces by passing a single inheritance restriction of class.

All fields declared inside an interface are public static final can't be changed. All Method signatures declared without

Instance Variable defined inside a class but outside any Method construct and codeblock

Abstract class

An abstract class is a class marked with an abstract key word that cannot be directly instantiated. It acts as a partial blueprint intended to be extended by concrete sub classes.

1. → It has no method body.
2. → Abstract Method must not contain curly braces.
3. → Enclosing class constraint.
- An abstract Method can only exist inside an abstract class or an Interface.

Local Variable & Global

Abstract class & Instance class

Instance Variable & Global Variable

3 → Mandatory Overriding

The first concrete subclass extending an abstract parent class must override and provide concrete implementation for all inherent abstract Methods.

Abstract cannot be declared as public, static, private, protected.

Public can be used like,

Public abstract class

It can't use public with class / concrete abstract public class.

Instance Variable

```

abstract class A {
    private X.Y;
}

```

Include datatype while declaring

Abstract (Variable) method

```

abstract class A {
    abstract X.Y;
}

```

Abstract Method.

Public abstract void main().
Public can be declared when an instance/object is not created.
If Any object is created, then the Abstract Method uses abstract void main() without public.

1. Sorter Strategy pattern - Merge Sort Program

2. Create an Abstract class shape that represents geometric shape.

abstract Method

class Name should be in Caps (Shape)
Abstract Method called double Area()
" " " double perimeter()
A concrete Method void display().

```
abstract class shape {
    abstract double area();
    abstract double perimeter();
}

class Circle extends shape {
    double r = 5;

    double area() {
        return 3.14 * r * r;
    }

    double perimeter() {
        return 2 * 3.14 * r;
    }

    void display() {
        System.out.println("Area = " + area());
        System.out.println("Perimeter = " + perimeter());
    }
}
```

```
}
}

public class Main {
    public static void main(String[] args) {
        Circle c = new Circle();
        c.display();
    }
}
```

Inheritance

Inheritance is a Mechanism where in a child class derives state and behaviour from a parent class using the extends keyword. It establishes and is-a, has-a relationship.

single - class B extends class A
Multiple - class C extends class B extends A
Hierarchical -

Upcasting & downcasting

Overloading

Multiple methods with same class name and different parameters.

```
class student {
    String name;
    student() {
    }
    student(String name) {
        this.name = name;
    }
}
```


Overriding
child class provides its own implementation of Method that is already defined in the parent class

this keyword represents current object.
this

Polymorphism

Greek - Many forms

It is the capability of a single entity (Method calling / interface references) to exhibit different behaviours depending on context or the underlying object type.

Types: Zero runtime.

- * Static - Compile time, Overloading
 - * Dynamic - Runtime, Overriding.
- } Binding time.

Usage (Parameter, type, number)

object() - stored in heap memory.
usage (pointers)

Compile time polymorphism - Method Overloading

- * Method Overloading occurs when a single class defines multiple Methods sharing the same name but having different parameters.
- * Overloading cannot be achieved solely (simply) by changing the return type of a Method.

Generally to add two numbers (Sum of numbers)
a =
b =
S.O.P (" " + a+b);

But using parameter constructor.
public int add (int a, int b)
{
 return (a+b);
}

Method Overload Overriding occurs when a child class provides an implementation of a Method, i.e. already declared in its parent class / interface.

- * When calling an overridden method through a parent reference Java evaluates the actual line on the heap at runtime rather than the reference time.

Write a program using Overriding Method.
With properties play and sound, with parent reference

→ Control flow (executes statement)

in which

statements execute it has sequential Execution (top to bottom)

```
int a = 10 - set      Top
int b = 10
int Sum = a+b
P.out (Sum)           bottom
```

→ Control flow

Control flow lets a program make decisions and execute different blocks of code based on conditions (true or false return value)

* If, If-else, else-if, Switch Case.

If Statement

It executes the block when condition is true

Syntax: ~~if~~ if (condition)
{
}
}

If-else Statement

Provides an alternative path when condition is false.

Syntax: if (condition)
{
 Statement
}
else {
 Statement
}

int marks = 45
if marks > 90
 s.o.p (Topper)
else {
 s.o.p (First class)
}

If-else-if ladder.

Checks Multiple condition in sequence only the 1st 2 block runs.

Syntax: if (condition) {
 }
else if (condition) {
 }
else if (condition) {
 }
else {
 out
}

Switch

Switch expression is evaluated once. the result is compared with each case value. It is used. Instead of Multiple if statements. If there is a match, the Matching block of code runs.

* The break statement stops the Switch after the matching case has run. The default statement runs if there is no match.

Loop

→ print odd and even number.

It executes block of code repeatedly based on the Condition. The number of iteration known in advance.

While loop

The No. of iteration is known before depends on the Condition. Exit the current Method.

Methods

A Method is a reusable block of Code that encapsulates a single responsibility.

Recursion

Recursion occurs when a method calls itself to solve a problem by working on a smaller instance of that same problem.

Java runtime uses a stack to track method calls.

Every call pushes a new program frame on to the stack.

When a method returns its frame is popped and control returns to the caller

Call Stack LIFO Void Stack () {} →

→ Generic

Generic lets classes, interfaces and methods operate on a type parameter specified by the caller giving compile time type safety and eliminating manual casting.

→ Collection framework

Set of classes and interfaces that provide readymade data structure to store and manipulate group of objects.

Collect is an object that can store multiple element as a single unit.

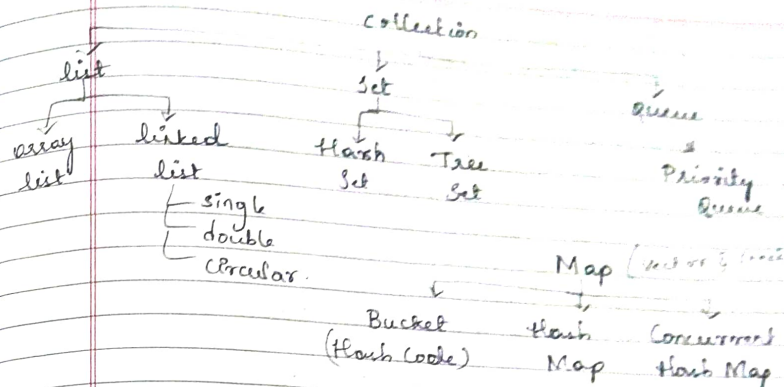
collection provides interfaces, defines what a data structure can do.

Syntax: List <patient>
= new ArrayList <>();

Actual classes that implement those interface is called implementation.

Mechanism for traversal collection is called iterators.

In collect we use set, list, queue.



Wrapper class

Is a class whose object wraps or encapsulate primitive data type.

byte <Byte>
int <Integer>
char <Character>
Float <Float>

List Interface

A List represents an ordered collection maintains insertion order, it allows duplicate supports index based access.

Elements can be accessed using indexes

ArrayList

It is the one of the most frequently used java collection in DSA.

An ArrayList is backed internally with dynamic array, unlike an normal array it can grow dynamically.

ArrayList <Integer> number.
= new ArrayList <>();

Write a program that Manages student marks using array list
score 85 92 78 90 65 88 76.

1. Display all the marks
2. Find ~~the~~ & display highest Marks
3. Display lowest Marks
4. Average Marks.
5. Remove the Marks below 80
6. Display Marks list
7. Marks 90 present in list (check)

int
double sum avg sum / Total

double average = int sum / marks.size(0);

Linked list

It implements three interfaces at once.
while in list, queue, dequeue.

Which is why it shows up in so many different roles. Internally it is a doubly linked list.
Each Node stores 3 things

- 1) A reference to the previous node
- 2) The data
- 3) Reference to the next node.

ArrayList

- * Dynamic array
- * fast
- * Less overload
- * Insertion, $O(n)$

LinkedList

Double linked list
slow (coz index)
Memory (more overload)
 $O(1)$

Set Interface

A set represents a collection of unique elements.
Set Interface extends the Collection Interface. It is available in the java.util package.
It does not allow duplicate elements.

HashSet

Use a HashSet when you need fast membership checking and uniqueness. It does not Order (data) automatically.

Tree Set

Stores unique Elements in Sorted order

Set < Integer > name =
new TreeSet <> ();

Queue Interface (FIFO)

offer() → add

poll() → Remove

peek() → Return first Element.

Map

as
Map stores data as key - Value pairs

Create a simple student result management system where student roll no is 3.

student name - marks or Value.

Performing adding, searching, displaying, removing, updating.

Tree Map

Keeps its keys in sorted order at all times. Like TreeSet implements Tree Map a navigable interface with the same family of

neighbour finding Methods and operating on keys

map.floorKey(25) -> highest element
map.ceilingKey() - lowest
map.firstKey() - first
map.lastKey() - last
map.headMap(10) - all element until 10
map.tailMap(5) - all elements from last to 5

Comparable

It defines a class single nature ordering implemented inside the class itself

Comparator

It defines an external swapping ordering useful when you need several different ordering for the same class.

Exception

An exception is an abnormal condition that occurs during the execution of a program and disturbs the normal expected flow of instructions

Throwable

Throwable is the superclass of all objects that can be thrown by Java exception.

Every error and every exception checked or unchecked ultimately extends Throwable

It provide methods such as:

getMessage()

toString()

getCause() -> SQL Exception

PrintStackTrace()

Develop a simple banking application that validates user inputs using exception

Leetcode 82, 83.

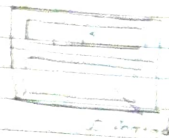
Multi threading
Thread.

Multithreading is the process of executing multiple threads concurrently within a single program.

Thread is the smallest unit of execution within a process.

UI thread, Network, Rendering thread.

public static Main - thread



Process Vs Thread.

A process is a running program with its own independent, isolated block of memory that other process cannot directly see into.

Ex: Chrome.

Thread pool

Thread is an execution path Executor Service within

If everything happens sequential the application may appear slow, the user is stuck waiting for the download to finish completely before processing even starts, and waiting for processing to finish before the UI updates at all, even though these are conceptually fairly independent activities.

Race Condition

A Race condition occurs when multiple threads access shared mutable data concurrently and the final result depends on the timing or order of their execution.

Deadlock

A deadlock occurs when two or more threads wait indefinitely for resources held by each other.

If the race condition is a case of threads interacting with shared data, a deadlock is a case of thread with each others.

Progress

Neither thread does anything wrong individually but together they lock the program permanently.

Synchronization

The Mechanism to control the access of multiple threads to share the resources. It is essential in multithreaded environment to prevent thread interference and consistency problems. Synchronization ensures that only one thread access the shared resources at a time.

Mixing a correct program answers only one question a second equally important question is how efficiently the program reaches that answer as the input ~~becomes~~ grows.

Complexity analysis is the discipline of answering that second question before you even run the code.

Ex: Binary Search & Linear Search.

Two measures of complexity.

→ Time Complexity
→ Space Complexity

Time Complexity is not literally how many seconds does it takes, it describes the growth rate of computational work relative to input size n .

Asymptotic Analysis

→ Worst - takes large time.
→ Best - lowest time.
→ Average - Average time.

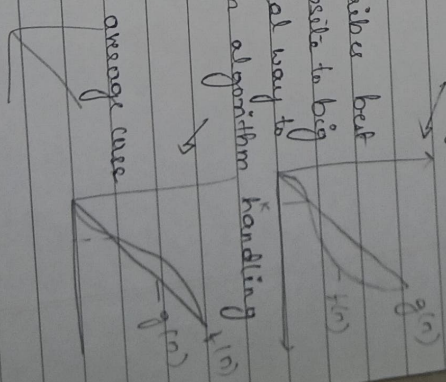
Calculated based on run time complexity.

Big O - This measures the performance of an algorithm by simply providing the order of growth of a function.

This notation provides upper bound of function which ensures that function never grows faster than the upper bound.

Omega - It basically describes best case scenario which is opposite to big O notation. It is the formal way to represent lower bound of an algorithm handling time.

Theta - It mainly describes average case.



Race Condition

A Race condition occurs when multiple threads access shared mutable data concurrently and the final result depends on the timing or order of their execution.

Deadlock

A deadlock occurs when two or more threads wait indefinitely for resources held by each other.

If the race condition is a case of threads interfering with sharing data, a deadlock is a case of thread with each others progress.

Neither thread does anything wrong individually but together they lock the program permanently.

Synchronization

The Mechanism to control the access of multiple threads to share the resources. It is essential in multithreaded environment to prevent thread interference and consistency problems.

Synchronization ensures that only one thread access the shared resources at a time.

Writing a correct program answers only one question a second equally important question is how efficiently the program reaches that answer as the input ~~size~~ grows.

Complexity analysis is the discipline of answering that second question before you even run the code.

Ex: Binary Search & Linear Search.

Two measures of complexity.

→ Time Complexity

→ Space Complexity

Time Complexity is not literally how many seconds does it takes. It describes the growth rate of computational work relative to input size.

Asymptotic Analysis

→ Worst - takes huge time.

→ Best - lowest time.

→ Average - Average time.

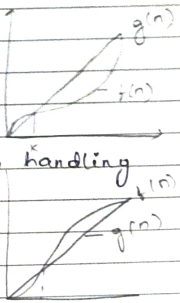
Calculated based on run time complexity

Big O - This Measures the performance of an algorithm by simply providing the order of growth of a function.

This notation provides upper bound of function which ensures that function never grows faster than the upper bound.

Omega - It basically describes best case scenario which is opposite to big O notation. It is the formal way to represent lower bound of an algorithm handling time.

Theta - It mainly describes average case.



Space Complexity

only a fixed number of extra variables are used regardless of n .

Additional spaces $O(1)$.

Given numbers. $num = [2, 7, 11, 15]$, $target = 9$.

Find the indexed of 2 numbers that add upto the target

Write the program to calculate $a^b \text{ Mod } M$ using modular exponential, values of a & b may be very large, so the program should avoid directly calculating a^b .

```
for i = -1; i <= 100; ++i {
    print("Hello World");
}
```

Time & Space Complexity.

```
for (int i = 1; i < nums.length; i++) {
    for (int j = i + 1; j < nums.length; j++) {
        if (nums[i] + nums[j] == target) {
            return new int[] {
                i, j
            };
        }
    }
}
```